

Adaptation Experiment 3: Learned Retrieval Gates

1 Candidates and targets

Two gate families are evaluated. A context gate chooses between the vanilla forecast V and the context-informed forecast C . An aggregated-neighbor gate chooses between V and

$$\hat{Y}_h = \sum_{j=1}^k w_j Y_{j,h}.$$

Write either candidate as $A \in \{C, \hat{Y}\}$. The horizon-level advantage used for supervision is

$$\Delta_h = (Y_h - V_h)^2 - (Y_h - A_h)^2.$$

A positive value means the candidate is better. Shared gates make one decision for the complete forecast window using $\bar{\Delta} = H^{-1} \sum_h \Delta_h$; horizon gates make H separate decisions.

Experimental goal. Learn when context or retrieved outcomes should replace the reliable vanilla forecast, and compare conditional gates with no-feature and target-aware upper bounds.

2 Engineered features

Let

$$\mu_w(z) = \sum_j w_j z_j, \quad \sigma_w(z) = \sqrt{\sum_j w_j (z_j - \mu_w(z))^2}.$$

For each neighbor define the across-horizon summaries

$$\begin{aligned} D_j^Y &= H^{-1} \sum_h (Y_{j,h} - V_h), & S_j^Y &= \text{sd}_h(Y_{j,h} - V_h), \\ D_j^N &= H^{-1} \sum_h (Y_{j,h} - N_{j,h}), & S_j^N &= \text{sd}_h(Y_{j,h} - N_{j,h}). \end{aligned}$$

Both gate shapes receive

$$\mu_w(D^Y), \sigma_w(D^Y), \mu_w(S^Y), \quad \mu_w(D^N), \sigma_w(D^N), \mu_w(S^N).$$

They also receive the query mean and standard deviation; the mean and between-neighbor standard deviation of neighbor look-back means and look-back standard deviations; same-user ratio; mean neighbor age; weight standard deviation and maximum; and mean retrieval distance.

The candidate features differ by gate shape:

- A shared gate receives $\text{mean}_h(A_h - V_h)$ and $\text{sd}_h(A_h - V_h)$.
- The model for horizon h receives only $A_h - V_h$, not the complete vector $(A_1 - V_1, \dots, A_H - V_H)$. It additionally receives the local $\mu_w(Y_{j,h} - V_h)$, $\sigma_w(Y_{j,h} - V_h)$, $\mu_w(Y_{j,h} - N_{j,h})$, and $\sigma_w(Y_{j,h} - N_{j,h})$.

Thus every horizon gate has the same feature count and there are H independent models; information from another candidate horizon cannot leak through the candidate-difference feature.

3 Models and artifact names

Name pattern	Meaning
<code>bayes_<A>_shared/horizon</code>	No-feature decision from the average $T_1 + T_2$ advantage.
<code>catboost_<A>_classifier_</code> <code>shared/horizon</code>	CatBoost estimate of $\mathbb{P}[\Delta > 0]$; choose A above probability 1/2.
<code>catboost_<A>_regressor_</code> <code>shared/horizon</code>	CatBoost estimate of the signed loss advantage; choose A above zero.
<code>oracle_<A>_shared/horizon</code>	Target-aware upper bound evaluated on the scored split; never deployable.

Here `<A>` is context or `aggr_y`.

4 Validation protocol

Extraction provides pooled $T_1 + T_2$ and fixed T_3 payloads. For these gates, the last 20% of pooled query dates is T_2 by default:

1. fit CatBoost on T_1 with at most 300 trees, learning rate 0.03, depth 4, and balanced classifier classes;
2. use T_2 only for early stopping and selection of the tree count;
3. instantiate a fresh model with that selected count and refit on all of $T_1 + T_2$;
4. score once on complete T_3 .

CatBoost does not require a separate hyperparameter search here: depth and learning rate are fixed experimental choices, while T_2 selects effective capacity through early stopping. Feature importance is saved separately for each candidate, objective, and decision shape.

The implementation fits horizons serially. For every gate it selects the tree count, releases the selection model, refits one final model, writes that model in CatBoost’s native format, scores disk-backed arrays, and releases the model before continuing. Gate summaries are computed in bounded row chunks and a single horizon feature matrix is materialized at a time. Predictions and diagnostic scores are separate NumPy arrays indexed by the sole current `prediction_manifest.json`; `gate_artifacts.json` indexes the models, and `result_manifest.json` is written last as the completion marker.

This refit is valid because C and $\hat{Y}$ are fixed candidates. For a future gate whose candidate is itself trained, the stage-two protocol is different: fit the candidate on T_1 and the gate on T_2 without refitting the candidate afterward. Refitting the candidate on $T_1 + T_2$ would change the gate inputs and make its T_2 supervision in-sample. Avoiding that bias requires out-of-fold candidate predictions or an additional split.

Launcher: `gates.slurm`

Module: `src.adaptors.baselines.evaluate -family gates`

5 Interpretation

The no-feature gate measures whether one candidate wins globally. CatBoost adds conditional decisions. The oracle gap is headroom, not performance. Context and aggregated-neighbor gates answer different questions and should both be reported against V on identical T_3 windows.